Replication Code File

This file accompanies "Who Pays for the Next Attempt? Contract Design and Escalation Rules for AI-Assisted Software Maintenance." It lists the code that produces every number, table, and figure in the manuscript. Identifying information has been removed for review.

Environment

Python 3.13 with numpy 2.5.3, scipy 1.18.1, pandas 3.0.5, pyarrow 26.0.0, and matplotlib 3.11.2. The scripts under `contracts/` import each other (`contracts.py` is imported by most), so place them in a single folder and run them from there. Random seeds are fixed inside each script.

Data and how to obtain it

- *RepairBench results.* Clone the public RepairBench repository and pass its `results` folder as the first argument of `load_repairbench.py`. We used the commit of 26 October 2025. The loader writes `bug_counts.json`, which `analyze.py` reads from the current directory.
- *SWE-bench submissions.* Clone the public SWE-bench experiments repository and pass its `evaluation` folder to `swebench_repo_level.py`. The script reads the resolved lists in each `results/results.json`.
- *HumanEvalFix.* Download the Python split of the HumanEvalPack dataset (Hugging Face, `bigcode/humanevalpack`, file `python/test-00000-of-00001.parquet`) and save it as `hep_python.parquet` next to `sample_humanevalfix.py`. Sampling calls the OpenRouter chat-completions API; set the environment variable `OPENROUTER_API_KEY` to your own key. Run `python3 sample_humanevalfix.py <model> 40` for each model (we used `meta-llama/llama-3.2-3b-instruct`, `mistralai/ministral-8b-2512`, and `google/gemma-3-4b-it`), then `python3 analyze_hef.py`. Generated code is not stored; only pass/fail outcomes and token counts are written. The sampling cost was $0.56.
- *Prices and statistics.* The calibration in `calibration.py` uses the figures cited in Section 5.4 (BLS wage, benefits share, SWE-bench Verified time buckets, OpenRouter prices, and published per-task agent costs) as constants, with the sources named in comments and in the manuscript.

Run order

1. `contracts.py` (equilibria for the three heterogeneity profiles), `robustness.py`, `running_example.py`, `incentive_rent_table.py`.
2. `asymmetric.py`, `asym_sweep.py`, `diff_bertrand.py`, `diff_sweep.py`.
3. `calibration.py`, `sensitivity.py`, `fig_sensitivity.py`.
4. `verify_proofs.py`, `general_theory.py`.
5. Empirical: `load_repairbench.py`, `analyze.py`, `swebench_repo_level.py`, `sample_humanevalfix.py`, `analyze_hef.py`.

Each script writes its output to a `results_*.txt` file, reproduced in Online Appendices E to H.

Where each result comes from

Manuscript item	Script
Table 1, Figure 1	`running_example.py`
Table 2	`incentive_rent_table.py`
Table 3 and the 296-draw comparison (Section 5.1)	`contracts.py`, `robustness.py`
Section 5.2	`asymmetric.py`, `asym_sweep.py`

Manuscript item	Script
Table 4 and the 240-cell sweep (Section 5.3)	`diff_bertrand.py`, `diff_sweep.py`
Section 5.4 calibration, Table 5, Figure 2	`calibration.py`, `sensitivity.py`, `fig_sensitivity.py`
Section 6, RepairBench	`load_repairbench.py`, `analyze.py`
Section 6, SWE-bench	`swebench_repo_level.py`
Section 6, HumanEvalFix	`sample_humanevalfix.py`, `analyze_hef.py`
Online Appendix D numerical checks	`verify_proofs.py`, `general_theory.py`

Code listings

contracts.py

Core model: market, contracts, first best and Stackelberg equilibria.

```
"""Stackelberg contract analysis: vendor (leader) sets a per-attempt
fee w or a success fee f; firm (follower) picks its attempt cap N.
Units: dollars. a,b: Beta heterogeneity; H: human fix cost; v: firm's
own cost per attempt (test run/triage); k: vendor marginal cost per
attempt."""
import numpy as np
from scipy.special import betaln

def surv(a, b, J): return np.exp(betaln(a, b + np.arange(J + 1)) -
betaln(a, b))   # S(0..J)
class Mkt:
    def __init__(s, a, b, H, v, k, J=4000):
        s.a, s.b, s.H, s.v, s.k = a, b, H, v, k; s.S = surv(a, b, J);
s.cum = np.concatenate([[0], np.cumsum(s.S)])
    def att(s, N): return s.cum[N]                  # E[min(T,N)]
    def P(s, N): return 1 - s.S[N]                  # P(solved within
N)
    def Nstar(s, ratio): return int(min(max(0, np.ceil(s.a*ratio - s.a
- s.b)), len(s.S)-1))   # ratio = (H or H-f)/(firm per-attempt cost)
    # --- contracts ---
    def per_attempt(s, w):
        N = s.Nstar(s.H/(s.w_c(w)));  c = w + s.v
        firm = c*s.att(N) + s.H*(1 - s.P(N)); vend = (w -
s.k)*s.att(N)
        return dict(N=N, firm=firm, vend=vend, welfare=s.H - firm +
vend - 0)   # welfare = total savings vs human (firm saving + vendor
profit)
    def w_c(s, w): return w + s.v
    def success_fee(s, f):
        N = s.Nstar((s.H - f)/s.v)
        firm = s.v*s.att(N) + f*s.P(N) + s.H*(1 - s.P(N)); vend =
f*s.P(N) - s.k*s.att(N)
        return dict(N=N, firm=firm, vend=vend, welfare=s.H - firm +
vend)
    def first_best(s):
        N = s.Nstar(s.H/(s.k + s.v)); cost = (s.k + s.v)*s.att(N) +
s.H*(1 - s.P(N))
        return dict(N=N, cost=cost, surplus=s.H - cost)

def solve(m, grid=2000):
    fb = m.first_best()
    ws = np.linspace(m.k, m.H, grid); best_w = max((m.per_attempt(w) |
dict(w=w) for w in ws if m.per_attempt(w)['firm'] <= m.H), key=lambda
r: r['vend'])
    fs = np.linspace(0, m.H, grid); best_f = max((m.success_fee(f) |
dict(f=f) for f in fs if m.success_fee(f)['firm'] <= m.H), key=lambda
r: r['vend'])
```

```
    f_eff = m.H*m.k/(m.k + m.v); eff = m.success_fee(f_eff) |
dict(f=f_eff)
    return fb, best_w, best_f, eff

if __name__ == "__main__":
    out = []; P = out.append
    scen = {"RepairBench-like (a=.12,b=.39)": (.12, .39), "SWE-
Verified-like (a=.25,b=.34)": (.25, .34), "Synthetic easy (a=.4,b=3)":
(.4, 3.0)}
    for name, (a, b) in scen.items():
        for (H, v, k) in [(100, 1.0, 0.2), (100, 1.0, 1.0), (300, 2.0,
0.5), (100, 0.2, 0.2)]:
            m = Mkt(a, b, H, v, k); fb, bw, bf, ef = solve(m)
            P(f"{name} | H={H} v={v} k={k}")
            P(f"   first-best: N={fb['N']:4d}
surplus={fb['surplus']:7.2f}")
            P(f"   per-attempt (monopoly w*={bw['w']:.2f}):
N={bw['N']:4d} vendor={bw['vend']:6.2f} firm_saving={m.H-
bw['firm']:6.2f} welfare={bw['welfare']:7.2f}
({100*bw['welfare']/fb['surplus']:.0f}% of FB)")
            P(f"   success-fee (monopoly f*={bf['f']:.2f}):
N={bf['N']:4d} vendor={bf['vend']:6.2f} firm_saving={m.H-
bf['firm']:6.2f} welfare={bf['welfare']:7.2f}
({100*bf['welfare']/fb['surplus']:.0f}% of FB)")
            P(f"   success-fee at efficient f=Hk/(k+v)={ef['f']:.2f}:
N={ef['N']:4d} vendor={ef['vend']:6.2f} firm_saving={m.H-
ef['firm']:6.2f} welfare={ef['welfare']:7.2f}")
            P("")
    open("results_contracts.txt", "w").write("\n".join(out)); print("\
n".join(out))
```

running_example.py

Table 1 and Figure 1 (running example).

```
import numpy as np
from scipy.special import betaln
from scipy.stats import beta as B
a, b, H, v, k = .12, .39, 100., 1., .2
J = 400; n = np.arange(J+1)
S = np.exp(betaln(a, b+n) - betaln(a, b)); E = np.concatenate([[0],
np.cumsum(S)])[:J+1]; P = 1 - S
q = a/(a+b+np.arange(J))
W = lambda N: H*P[N] - (k+v)*E[N]
G = lambda N: v*(P[N]/q[N-1] - E[N]); R = lambda N: W(N) - G(N)
phi = lambda N: H - v/q[N-1]
out = []; Pr = out.append
Pr(f"Beta({a},{b}); H={H}, v={v}, k={k}; prior mean success
{a/(a+b):.3f}")
Pr("q_n (success prob of attempt n+1 after n failures): " + ",
".join(f"n={i}: {q[i]:.4f}" for i in (0,1,2,3,4,5,9,10,19,29)))
Pr(f"posterior mean after 10 failures {q[10]:.4f}; after 30 failures
{q[30]:.4f}")
Pr("fraction of defects with p<0.01: %.2f ; p<0.05: %.2f ; p>0.5: %.2f
; p>0.9: %.2f" % (B.cdf(.01,a,b), B.cdf(.05,a,b), 1-B.cdf(.5,a,b), 1-
B.cdf(.9,a,b)))
Pr(f"marginal value q_n*H vs efficient cost k+v = {k+v}: attempt 10
(n=9): {q[9]*H:.2f}; attempt 11 (n=10): {q[10]*H:.2f}")
Pr("N | P(N) | E(N) | W(N) | G_min(N) | R(N) | phi_(N-1)")
for N in (1,2,3,5,10,20):
    Pr(f"{N:2d} | {P[N]:.3f} | {E[N]:.2f} | {W(N):.2f} | {G(N):.2f} |
{R(N):.2f} | {phi(N):.1f}")
Pr(f"per-attempt monopoly price q_0 H - v = {q[0]*H - v:.2f}; success
fee at step 3: phi_2 = {phi(3):.1f}")
Pr(f"human-only per 1,000 defects: ${H*1000:,.0f}; first best N=10:
W=${W(10)*1000:,.0f}")
open("results_running_example.txt", "w").write("\n".join(out));
print("\n".join(out))
# figure
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as
plt
plt.rcParams.update({"font.size": 8.5, "font.family": "serif"})
fig, ax = plt.subplots(1, 2, figsize=(6.5, 2.9),
constrained_layout=True)
nn = np.arange(0, 31)
ax[0].semilogy(nn, q[nn]*H, "k-o", ms=3, lw=1, label="Marginal value
of the next attempt, $q_n H$")
ax[0].axhline(k+v, color="0.4", ls="--", lw=1, label="Efficient cost
$k+v$ (first best: 10 attempts)")
ax[0].axhline(q[0]*H, color="0.7", ls=":", lw=1)
ax[0].set_xlabel("failures so far, $n$"); ax[0].set_ylabel("dollars");
ax[0].set_title("(a) Failures are bad news", fontsize=9, loc="left")
```

```
ax[0].legend(fontsize=6.5, loc="upper right", frameon=False)
Ns = [1, 2, 3, 5, 10]; Rv = [R(N) for N in Ns]; Gv = [G(N) for N in
Ns]
x = np.arange(len(Ns)); ax[1].bar(x, Rv, color="0.25", label="Most the
vendor can keep, $R(N)$"); ax[1].bar(x, Gv, bottom=Rv, color="0.75",
label="Rent the firm must keep, $G_{min}(N)$")
ax[1].set_xticks(x); ax[1].set_xticklabels([f"N={N}" for N in Ns]);
ax[1].set_ylabel("dollars per defect"); ax[1].set_title("(b) Surplus
$W(N)$ and its split", fontsize=9, loc="left")
ax[1].set_ylim(0, 48); ax[1].legend(fontsize=6.5, loc="upper left",
frameon=False)
fig.savefig("fig_intuition.png", dpi=220); print("saved")
```

incentive_rent_table.py

Table 2 (incentive rent).

```
"""Table 2 of the manuscript: incentive rent at the first-best depth
(H=100, v=1, k=0.2)."""
import numpy as np
from scipy.special import betaln
H, v, k = 100., 1., .2
out = []; P = out.append
P("Heterogeneity (a,b) | N^FB | W(N^FB) | G_min(N^FB) | R(N^FB) |
Rent/W(N^FB) | vendor-optimal depth n_hat, R(n_hat)")
for name, (a, b) in {"RepairBench-like": (.12, .39), "SWE-Verified-
like": (.25, .34), "Synthetic": (.40, 3.0)}.items():
    J = 600; n = np.arange(J+1)
    S = np.exp(betaln(a, b+n) - betaln(a, b)); E =
np.concatenate([[0], np.cumsum(S)])[:J+1]; Pn = 1 - S
    q = a/(a+b+np.arange(J))
    W = lambda N: H*Pn[N] - (k+v)*E[N]
    G = lambda N: v*(Pn[N]/q[N-1] - E[N]); R = lambda N: W(N) - G(N)
    nfb = int(np.argmax([W(N) for N in range(J)]))
    nhat = max(range(1, J), key=R)
    P(f"{name} ({a},{b}) | {nfb} | {W(nfb):.1f} | {G(nfb):.1f} |
{R(nfb):.1f} | {100*G(nfb)/W(nfb):.0f}% | {nhat}, {R(nhat):.1f}")
open("results_incentive_rent.txt", "w").write("\n".join(out));
print("\n".join(out))
```

robustness.py

Random-draw comparison of the contracts (Section 5.1).

```
import numpy as np
from contracts import Mkt, solve
rng = np.random.default_rng(7)
out = []; P = out.append
# (1) random parameter sweep
rows = []
for _ in range(300):
    a = rng.uniform(.05, .6); b = np.exp(rng.uniform(np.log(.1),
np.log(4))); H = np.exp(rng.uniform(np.log(30), np.log(500)))
    v = H*np.exp(rng.uniform(np.log(.001), np.log(.03))); k =
v*np.exp(rng.uniform(np.log(.1), np.log(3)))
    m = Mkt(a, b, H, v, k, J=3000); fb, bw, bf, ef = solve(m,
grid=400)
    if fb['surplus'] <= 1e-6: continue
    rows.append(dict(a=a, b=b, ratio=H/v,
W_pa=bw['welfare']/fb['surplus'], W_sf=bf['welfare']/fb['surplus'],
V_pa=bw['vend'], V_sf=bf['vend'],
                     F_pa=m.H-bw['firm'], F_sf=m.H-bf['firm'],
N_fb=fb['N'], N_pa=bw['N'], N_sf=bf['N'],
vshare_eff=ef['vend']/fb['surplus'], k_over_kv=k/(k+v)))
R = rows; n = len(R)
f = lambda key: np.array([r[key] for r in R])
P(f"Random sweep: {n} parameter draws (a~U(.05,.6), b log-U(.1,4), H
log-U(30,500), v/H log-U(.001,.03), k/v log-U(.1,3))")
P(f"Vendor profit: success-fee >= per-attempt in {np.mean(f('V_sf') >=
f('V_pa')-1e-9)*100:.0f}% of draws")
P(f"Firm saving:   success-fee >= per-attempt in {np.mean(f('F_sf') >=
f('F_pa')-1e-9)*100:.0f}% of draws")
P(f"Welfare/FB:    per-attempt median {np.median(f('W_pa')):.2f} [IQR
{np.percentile(f('W_pa'),25):.2f}-{np.percentile(f('W_pa'),75):.2f}];
success-fee median {np.median(f('W_sf')):.2f} [IQR
{np.percentile(f('W_sf'),25):.2f}-{np.percentile(f('W_sf'),75):.2f}]")
P(f"Success-fee welfare > per-attempt welfare in {np.mean(f('W_sf') >=
f('W_pa')-1e-9)*100:.0f}% of draws")
P(f"Median attempts: first-best {np.median(f('N_fb')):.0f}, per-
attempt monopoly {np.median(f('N_pa')):.0f}, success-fee monopoly
{np.median(f('N_sf')):.0f}")
P(f"Efficient-fee vendor share of surplus equals k/(k+v): max abs diff
{np.max(np.abs(f('vshare_eff')-f('k_over_kv'))):.3f}")
# (2) quality-misperception regret: firm believes a_believed, true
a_true (b fixed). Cap chosen from belief; contract terms fixed at
belief-based monopoly price.
P("")
P("Regret from over-estimating vendor quality (H=100, v=1, k=0.2,
b=0.39): firm saving vs human-only when it sets N from believed 'a'
but true 'a' is lower")
P("a_true   contract         saving(correct belief a=0.25 vs true)
```

```
saving(believed a=0.25)   loss")
b, H, v, k = .39, 100., 1., .2
belief = Mkt(.25, b, H, v, k); fb_, bw_, bf_, ef_ = solve(belief,
grid=800)
w0, f0 = bw_['w'], ef_['f']
for at in [.25, .15, .10, .05]:
    true = Mkt(at, b, H, v, k)
    # fee levels fixed by contract: per-attempt at w=k+0.5 (cost-plus)
; success fee at efficient f
    w_cp = 2.0
    for name, fn, term in [("per-attempt w=2.0", true.per_attempt,
w_cp), ("success fee f=Hk/(k+v)", true.success_fee, f0)]:
        # firm sets N from believed a under the same terms
        Nb = belief.Nstar(H/(term+v)) if "per-attempt" in name else
belief.Nstar((H-term)/v)
        if "per-attempt" in name: c = term+v; cost_b = c*true.att(Nb)
+ H*(1-true.P(Nb)); Nt = true.Nstar(H/c); cost_t = c*true.att(Nt)
+H*(1-true.P(Nt))
        else: cost_b = v*true.att(Nb) + term*true.P(Nb) + H*(1-
true.P(Nb)); Nt = true.Nstar((H-term)/v); cost_t = v*true.att(Nt)
+term*true.P(Nt)+H*(1-true.P(Nt))
        P(f"{at:5.2f}    {name:22s} {H-cost_t:8.2f}   {H-cost_b:8.2f}
{cost_b-cost_t:7.2f}   (N_believed={Nb}, N_true={Nt})")
open("results_robustness.txt", "w").write("\n".join(out)); print("\
n".join(out))
```

asymmetric.py

Private vendor quality: profit order and equilibria (Section 5.2).

```
"""Vendor knows its quality a_theta (high/low, common b); firm does
not. Contracts: per-attempt w (A) or success fee f (B).
Checks (i) profit-order reversal across contract types, (ii)
separating (Riley-type) and pooling outcomes."""
import numpy as np
from scipy.special import betaln
J = 1500
GRID = 300
class Typ:
    def __init__(s, a, b): s.a, s.b = a, b; s.S = np.exp(betaln(a,
b+np.arange(J+1)) - betaln(a, b)); s.cum = np.concatenate([[0],
np.cumsum(s.S)])
    def E(s, N): return s.cum[N]
    def P(s, N): return 1 - s.S[N]
def mix_q(mu, TH, TL):                # posterior predictive success
after n failures under belief mu on high type
    n = np.arange(J)
    wH = mu*TH.S[:J]; wL = (1-mu)*TL.S[:J]
    qH = TH.a/(TH.a+TH.b+n); qL = TL.a/(TL.a+TL.b+n)
    return (wH*qH + wL*qL)/(wH + wL)
def N_firm(q, X, Y):                  # continue at n iff q_n*X > Y ; q
decreasing
    idx = np.nonzero(q*X <= Y)[0]; return int(idx[0]) if len(idx) else
J-1
class Game:
    def __init__(s, aH, aL, b, H, v, k, mu):
        s.TH, s.TL, s.H, s.v, s.k, s.mu = Typ(aH, b), Typ(aL, b), H,
v, k, mu
    def N(s, kind, price, belief_mu):            # firm's budget given
contract and belief (belief_mu = P(high))
        q = mix_q(belief_mu, s.TH, s.TL)
        return N_firm(q, s.H, price + s.v) if kind == "A" else
N_firm(q, s.H - price, s.v)
    def prof(s, T, kind, price, N): return (price - s.k)*T.E(N) if
kind == "A" else price*T.P(N) - s.k*T.E(N)
    def firm_cost(s, T, kind, price, N):
        if kind == "A": return (price + s.v)*T.E(N) + s.H*(1 - T.P(N))
        return s.v*T.E(N) + price*T.P(N) + s.H*(1 - T.P(N))
    def grid(s, kind): return np.linspace(s.k if kind == "A" else 0,
s.H, GRID)
    def FI(s, T, belief):                          # vendor's best
contract when firm's belief is 'belief' (1=high,0=low); firm
participation evaluated at believed type
        best = None
        for kind in "AB":
            for p in s.grid(kind):
                N = s.N(kind, p, belief)
```

```
                Tb = s.TH if belief == 1 else s.TL
                if s.firm_cost(Tb, kind, p, N) > s.H: continue      #
firm rejects if believed cost exceeds human-only
                pr = s.prof(T, kind, p, N)
                if best is None or pr > best[0]: best = (pr, kind, p,
N)
        return best
if __name__ == "__main__":
    out = []; Pr = out.append
    aH, aL, b, H, v, k, mu = .25, .08, .35, 100., 1., .2, .5
    g = Game(aH, aL, b, H, v, k, mu)
    # (i) profit order for a common budget N
    Pr("(i) Profit order for a common attempt budget N (b=.35, H=100,
k=.2): per-attempt w=10  /  success fee f=50")
    for N in [1, 3, 10, 30]:
        Pr(f"  N={N:3d}: A: low {g.prof(g.TL,'A',10,N):6.2f} high
{g.prof(g.TH,'A',10,N):6.2f} | B: low {g.prof(g.TL,'B',50,N):6.2f}
high {g.prof(g.TH,'B',50,N):6.2f}")
    # (ii) full information
    fiH, fiL = g.FI(g.TH, 1), g.FI(g.TL, 0)
    Pr(f"\n(ii) Full-information optimum  high: profit {fiH[0]:.2f}
via {fiH[1]} price {fiH[2]:.1f} N={fiH[3]}   low: profit {fiL[0]:.2f}
via {fiL[1]} price {fiL[2]:.1f} N={fiL[3]}")
    # treated-as-low payoff for high type = what it earns if firm
believes low
    tl = g.FI(g.TH, 0); Pr(f"     High type if believed low: profit
{tl[0]:.2f} ({tl[1]} price {tl[2]:.1f}, N={tl[3]})")
    # separating: H picks contract (kind,p) with belief=1 s.t. L's
mimic profit <= L's FI profit
    best = None; rows = []
    for kind in "AB":
        for p in g.grid(kind):
            N = g.N(kind, p, 1)
            if g.firm_cost(g.TH, kind, p, N) > g.H: continue
            ph = g.prof(g.TH, kind, p, N); pl = g.prof(g.TL, kind, p,
N)
            if pl <= fiL[0] + 1e-9 and ph >= tl[0] - 1e-9:
                if best is None or ph > best[0]: best = (ph, kind, p,
N, pl)
    if best is None:
        Pr("\nSeparating (Riley-type): NO linear contract satisfies
both the low type's no-mimicry and the high type's participation-in-
separation constraints")
        best = (tl[0], tl[1], tl[2], tl[3], 0.0)
    else:
        Pr(f"\nSeparating (Riley-type): high type's best IC contract:
{best[1]} price {best[2]:.1f}, N={best[3]}, high profit {best[0]:.2f}
(FI {fiH[0]:.2f}), low mimic profit {best[4]:.2f} <= low FI
{fiL[0]:.2f}")
    # pooling at prior: single contract maximizing expected vendor
```

```
profit with belief=mu
    bp = None
    for kind in "AB":
        for p in g.grid(kind):
            N = g.N(kind, p, mu); Tm = None
            cost_m = mu*g.firm_cost(g.TH, kind, p, N) + (1-
mu)*g.firm_cost(g.TL, kind, p, N)
            if cost_m > g.H: continue
            ev = mu*g.prof(g.TH, kind, p, N) + (1-mu)*g.prof(g.TL,
kind, p, N)
            if bp is None or ev > bp[0]: bp = (ev, kind, p, N,
g.prof(g.TH, kind, p, N), g.prof(g.TL, kind, p, N))
    Pr(f"Pooling benchmark: expected-profit-maximizing single contract
{bp[1]} price {bp[2]:.1f}, N={bp[3]}, profits high {bp[4]:.2f} low
{bp[5]:.2f}")
    # welfare comparison (surplus = H - firm cost + vendor profit),
per type
    def W(T, kind, p, N): return g.H - g.firm_cost(T, kind, p, N) +
g.prof(T, kind, p, N)
    Pr("\nWelfare (surplus vs human-only) by true type:")
    Pr(f"  full info:  high {W(g.TH,fiH[1],fiH[2],fiH[3]):.2f}  low
{W(g.TL,fiL[1],fiL[2],fiL[3]):.2f}")
    Pr(f"  separating: high {W(g.TH,best[1],best[2],best[3]):.2f}  low
{W(g.TL,fiL[1],fiL[2],fiL[3]):.2f}")
    Pr(f"  pooling:    high {W(g.TH,bp[1],bp[2],bp[3]):.2f}  low
{W(g.TL,bp[1],bp[2],bp[3]):.2f}")
    Pr("  firm saving (H - cost): " + f"full info high {g.H-
g.firm_cost(g.TH,fiH[1],fiH[2],fiH[3]):.2f}, low {g.H-
g.firm_cost(g.TL,fiL[1],fiL[2],fiL[3]):.2f}; separating high {g.H-
g.firm_cost(g.TH,best[1],best[2],best[3]):.2f}; pooling high {g.H-
g.firm_cost(g.TH,bp[1],bp[2],bp[3]):.2f} low {g.H-
g.firm_cost(g.TL,bp[1],bp[2],bp[3]):.2f}")
    open("results_asymmetric.txt", "w").write("\n".join(out));
print("\n".join(out))

    # ---- sweep: does a separating linear contract exist? ----
    rng = np.random.default_rng(3); cnt = dict(n=0, sep=0, sepB=0,
A_profit_low_ge=0, ordB=0)
    for _ in range(60):
        aL_ = rng.uniform(.03, .2); aH_ = aL_ + rng.uniform(.08, .35);
b_ = np.exp(rng.uniform(np.log(.15), np.log(2)))
        H_ = np.exp(rng.uniform(np.log(50), np.log(300))); v_ =
H_*np.exp(rng.uniform(np.log(.003), np.log(.02))); k_ =
v_*np.exp(rng.uniform(np.log(.2), np.log(2)))
        gg = Game(aH_, aL_, b_, H_, v_, k_, .5)
        fl = gg.FI(gg.TL, 0); t_l = gg.FI(gg.TH, 0)
        if fl is None or t_l is None: continue
        cnt["n"] += 1; ok = None
        for kind in "AB":
            for p in gg.grid(kind):
```

```
                N = gg.N(kind, p, 1)
                if gg.firm_cost(gg.TH, kind, p, N) > gg.H: continue
                ph = gg.prof(gg.TH, kind, p, N); pl = gg.prof(gg.TL,
kind, p, N)
                if pl <= fl[0] + 1e-9 and ph > t_l[0] + 1e-6: ok =
(kind, p)
        if ok: cnt["sep"] += 1; cnt["sepB"] += ok[0] == "B"
    print("\nSweep (60 draws): separating linear contract exists in",
cnt["sep"], "of", cnt["n"], "draws; of which success-fee:",
cnt["sepB"])
    open("results_asymmetric.txt", "a").write(f"\nSweep: separating
linear contract exists in {cnt['sep']} of {cnt['n']} draws (success-
fee type in {cnt['sepB']})\n")
```

asym_sweep.py

Private vendor quality: sweep (Section 5.2).

```
import numpy as np
from asymmetric import Game
rng = np.random.default_rng(3)
def sep_best(g, fl, t_l):
    best = None
    for kind in "AB":
        for p in g.grid(kind):
            N = g.N(kind, p, 1)
            if g.firm_cost(g.TH, kind, p, N) > g.H: continue
            ph = g.prof(g.TH, kind, p, N); pl = g.prof(g.TL, kind, p,
N)
            if pl <= fl[0] + 1e-9 and ph > t_l[0] + 1e-6 and (best is
None or ph > best[0]): best = (ph, kind, p, N, pl)
    return best
rows = []
for i in range(60):
    aL = rng.uniform(.03, .2); aH = aL + rng.uniform(.08, .35); b =
np.exp(rng.uniform(np.log(.15), np.log(2)))
    H = np.exp(rng.uniform(np.log(50), np.log(300))); v =
H*np.exp(rng.uniform(np.log(.003), np.log(.02))); k =
v*np.exp(rng.uniform(np.log(.2), np.log(2)))
    g = Game(aH, aL, b, H, v, k, .5); fh, fl, t_l = g.FI(g.TH, 1),
g.FI(g.TL, 0), g.FI(g.TH, 0)
    if None in (fh, fl, t_l): continue
    sb = sep_best(g, fl, t_l)
    # does low mimic the H full-info contract? (IC_L slack at FI?)
    N_fi = g.N(fh[1], fh[2], 1); mim = g.prof(g.TL, fh[1], fh[2],
N_fi)
    W=lambda T,kind,p,N: g.H-g.firm_cost(T,kind,p,N)
+g.prof(T,kind,p,N)
    Wfi=W(g.TH,fh[1],fh[2],fh[3]); Wsep=(W(g.TH,sb[1],sb[2],sb[3]) if
sb else None)
    rows.append(dict(wsep=(Wsep/Wfi if sb else None), fsav_fi=g.H-
g.firm_cost(g.TH,fh[1],fh[2],fh[3]), fsav_sep=(g.H-
g.firm_cost(g.TH,sb[1],sb[2],sb[3]) if sb else None), aH=aH, aL=aL,
b=b, H=H, sep=sb is not None, sepkind=(sb[1] if sb else None),
distort=(fh[0]-sb[0])/fh[0] if sb else None,
                    mimic_fi=mim > fl[0] + 1e-9, N_fh=fh[3],
N_sep=(sb[3] if sb else None), kind_fh=fh[1], kind_fl=fl[1],
gain_vs_tl=((sb[0]-t_l[0])/t_l[0] if sb else None)))
n = len(rows); sep = [r for r in rows if r['sep']]
print(f"draws {n}; separating linear contract exists: {len(sep)}
({100*len(sep)/n:.0f}%); kind(s): {set(r['sepkind'] for r in sep)}")
print(f"FI contract of high type chosen from: {set(r['kind_fh'] for r
in rows)}; low type: {set(r['kind_fl'] for r in rows)}")
print(f"Low type would mimic H's full-info contract (IC_L violated) in
{100*np.mean([r['mimic_fi'] for r in rows]):.0f}% of draws")
```

```
d = [r['distort'] for r in sep]; print(f"H-type profit loss from
separation vs FI: median {100*np.median(d):.1f}% (IQR
{100*np.percentile(d,25):.1f}-{100*np.percentile(d,75):.1f}%)")
print(f"H-type gain from separating vs being treated as low: median
{100*np.median([r['gain_vs_tl'] for r in sep]):.1f}%")
ex = [r for r in sep if r['distort'] > 1e-6][:1] or sep[:1]
print("example:", {k: (round(v,3) if isinstance(v,float) else v) for
k, v in ex[0].items()})
import json; open("results_asym_sweep.txt","w").write(f"draws {n};
separating exists {len(sep)}; median distortion {100*np.median(d):.1f}
%; low would mimic FI contract in {100*np.mean([r['mimic_fi'] for r in
rows]):.0f}%\n")

w=[r['wsep'] for r in sep]; print(f"Welfare of high type under
separation relative to full information: median {np.median(w):.2f}
(share of draws with higher welfare: {100*np.mean([x>1 for x in
w]):.0f}%)")
fs=[r['fsav_sep']-r['fsav_fi'] for r in sep]; print(f"Firm saving
(high type) separation minus FI: median {np.median(fs):.2f}; firm
better off in {100*np.mean([x>0 for x in fs]):.0f}% of draws")
print(f"Attempt budget N_sep >= N_FI in
{100*np.mean([r['N_sep']>=r['N_fh'] for r in sep]):.0f}% of draws")
open("results_asym_sweep.txt","a").write(f"welfare sep/FI median
{np.median(w):.2f}; share higher {100*np.mean([x>1 for x in w]):.0f}%;
firm better off in {100*np.mean([x>0 for x in fs]):.0f}%; N_sep>=N_FI
in {100*np.mean([r['N_sep']>=r['N_fh'] for r in sep]):.0f}%\n")
```

diff_bertrand.py

Differentiated-Bertrand competition (Section 5.3, Table 4).

```
"""Differentiated Bertrand (logit) competition among n vendors for a
population of firms.
Firm i's utility from vendor j's contract = (cost saving vs human-only
at the firm's optimal attempt cap) + sigma*eps_ij ; outside option
(human only) has saving 0.
sigma = degree of differentiation (sigma->0: homogeneous Bertrand;
large sigma: strong vendor market power). Vendors choose the price of
a given contract form (per-attempt fee w or success fee f)."""
import numpy as np
from contracts import Mkt
GRID = 500
def curves(m, kind):
    ps = np.linspace(m.k if kind == "A" else 0.0, m.H, GRID)
    F = np.zeros(GRID); Pi = np.zeros(GRID); N = np.zeros(GRID, int);
Wm = np.zeros(GRID)
    for i, p in enumerate(ps):
        r = m.per_attempt(p) if kind == "A" else m.success_fee(p)
        F[i] = m.H - r['firm']; Pi[i] = r['vend']; N[i] = r['N'];
Wm[i] = r['welfare']
    return ps, F, Pi, N, Wm
def share(F_own, F_riv, n, sig):          # logit with outside option
(human-only, saving 0)
    a = F_own/sig; b = F_riv/sig; mx = max(a, b, 0.0)
    num = np.exp(a - mx); den = np.exp(-mx) + num + (n-1)*np.exp(b -
mx)
    return num/den
def share_vec(F_own, F_riv, n, sig):
    a = F_own/sig; b = F_riv/sig; mx = np.maximum(np.maximum(a, b),
0.0)
    num = np.exp(a - mx); den = np.exp(-mx) + num + (n-1)*np.exp(b -
mx); return num/den
def best_response(cv, F_riv, n, sig):
    ps, F, Pi, N, Wm = cv
    prof = share_vec(F, F_riv, n, sig)*Pi; i = int(np.argmax(prof));
return i, prof[i]
def sym_eq(cv, n, sig):
    ps, F, Pi, N, Wm = cv; best = None
    for j in range(len(ps)):              # candidate rival price index
        i, _ = best_response(cv, F[j], n, sig); gap = abs(ps[i] -
ps[j])
        if best is None or gap < best[0]: best = (gap, j, i)
    gap, j, i = best; return j if gap < 1e-9 else i, gap     # take
the best-response index when fixed point not exact
def summarize(m, kind, n, sig, cv):
    ps, F, Pi, N, Wm = cv; j, gap = sym_eq(cv, n, sig)
    # symmetric outcome at price index j
    s = share(F[j], F[j], n, sig); cs = sig*np.log(1 +
```

```
n*np.exp(F[j]/sig)); prof_total = n*s*Pi[j]
    return dict(p=ps[j], N=N[j], cover=n*s, F=F[j], Pi=Pi[j],
vend_total=prof_total, TW=cs + prof_total, gap=gap, idx=j)
def deviation_gain(m, eq, cvs, kind_from, kind_to, n, sig):
    """Profit of a vendor deviating to the other form against rivals'
equilibrium contract."""
    cvA = cvs[kind_from]; Fr = cvA[1][eq['idx']]
    cvB = cvs[kind_to]; i, prof = best_response(cvB, Fr, n, sig)
    s_eq = share(cvA[1][eq['idx']], Fr, n, sig)*cvA[2][eq['idx']]
    return prof, s_eq
if __name__ == "__main__":
    out = []; P = out.append
    m = Mkt(.12, .39, 100., 1., .2); cvs = {k: curves(m, k) for k in
"AB"}; fb = m.first_best()
    P(f"RepairBench-like (a=.12,b=.39), H=100 v=1 k=0.2; first-best
N={fb['N']} S^FB={fb['surplus']:.1f}")
    for sig in [1., 3., 10., 30.]:
        TWfb_n = lambda n: sig*np.log(1 + n*np.exp(fb['surplus']/sig))
        P(f"\n sigma = {sig}")
        P("  n | form |  price   N  coverage  firm saving  vendor
profit(per firm)  total vendor profit | welfare/FB | deviation to
other form profitable?")
        for n in [1, 2, 3, 5, 10]:
            res = {}
            for kind in "AB":
                res[kind] = summarize(m, kind, n, sig, cvs[kind])
            for kind in "AB":
                r = res[kind]; other = "B" if kind == "A" else "A"
                dev, base = deviation_gain(m, r, cvs, kind, other, n,
sig)
                P(f"  {n:2d} |  {kind}   | {r['p']:7.2f} {r['N']:3d}
{r['cover']:6.2f}   {r['F']:8.2f}    {r['Pi']:8.2f}
{r['vend_total']:8.3f}       | {r['TW']/TWfb_n(n):6.3f}    | {'yes' if
dev > base*(1+1e-6) + 1e-9 else 'no '} (dev {dev:.3f} vs eq
{base:.3f})")
    open("results_diff_bertrand.txt", "w").write("\n".join(out));
print("\n".join(out))
```

diff_sweep.py

Differentiated-Bertrand sweep (Section 5.3).

```
import numpy as np
from contracts import Mkt
from diff_bertrand import curves, summarize, deviation_gain, share
rng = np.random.default_rng(5); H=None
rows = []; gaps = []
for d in range(40):
    a = rng.uniform(.05, .5); b = np.exp(rng.uniform(np.log(.15),
np.log(3))); Hh = np.exp(rng.uniform(np.log(50), np.log(300)))
    v = Hh*np.exp(rng.uniform(np.log(.003), np.log(.02))); k =
v*np.exp(rng.uniform(np.log(.2), np.log(2)))
    m = Mkt(a, b, Hh, v, k, J=3000); fb = m.first_best()
    if fb['surplus'] < 1: continue
    cvs = {kk: curves(m, kk) for kk in "AB"}
    for n in [2, 5]:
        for sf in [.03, .1, .3]:
            sig = sf*fb['surplus']              # differentiation
scaled to first-best surplus
            res = {kk: summarize(m, kk, n, sig, cvs[kk]) for kk in
"AB"}
            step = Hh/500
            devA = deviation_gain(m, res["A"], cvs, "A", "B", n, sig);
devB = deviation_gain(m, res["B"], cvs, "B", "A", n, sig)
            TWfb = sig*np.log(1 + n*np.exp(fb['surplus']/sig))
            rows.append(dict(n=n, sf=sf, A_unstable=devA[0] >
devA[1]*(1+1e-6)+1e-9, B_stable=devB[0] <= devB[1]*(1+1e-6)+1e-9,
                             wA=res["A"]['TW']/TWfb, wB=res["B"]
['TW']/TWfb, NA=res["A"]['N']/max(fb['N'],1), NB=res["B"]
['N']/max(fb['N'],1), vendA=res["A"]['vend_total'], vendB=res["B"]
['vend_total'], FA=res["A"]['F'], FB=res["B"]['F'],
                             gapok=max(res['A']['gap'], res['B']
['gap']) <= 2*step))
out = []; P = out.append
P(f"Sweep: {len(rows)} (parameter draw x n x differentiation) cells;
sigma = {{0.03,0.1,0.3}} x first-best surplus; n in {{2,5}}")
P(f"Fixed-point accuracy within two price-grid steps in
{100*np.mean([r['gapok'] for r in rows]):.0f}% of cells")
P(f"Per-attempt equilibrium NOT deviation-proof (a vendor gains by
switching to a success fee): {100*np.mean([r['A_unstable'] for r in
rows]):.0f}% of cells")
P(f"Success-fee equilibrium deviation-proof against a switch to per-
attempt: {100*np.mean([r['B_stable'] for r in rows]):.0f}% of cells")
for n in [2, 5]:
    for sf in [.03, .1, .3]:
        R = [r for r in rows if r['n'] == n and r['sf'] == sf]
        P(f"  n={n} sigma={sf:4.2f}S: welfare/FB per-attempt
{np.median([r['wA'] for r in R]):.2f}, success fee {np.median([r['wB']
for r in R]):.2f}; budget/FB-depth A {np.median([r['NA'] for r in
```

```
R]):.2f}, B {np.median([r['NB'] for r in R]):.2f}; vendor profit B>=A
in {100*np.mean([r['vendB']>=r['vendA'] for r in R]):.0f}%; firm
saving A>B in {100*np.mean([r['FA']>r['FB'] for r in R]):.0f}%  (cells
{len(R)})")
open("results_diff_sweep.txt", "w").write("\n".join(out)); print("\
n".join(out))
```

calibration.py

Cost calibration from public sources (Section 5.4).

```
"""Calibration of H, v, k from public sources and sensitivity of the
main conclusions.
Sources (accessed 2026-10-25): BLS OOH software developers (May 2025):
median $64.44/h wage ($134,040/yr in Quick Facts; $135,980 May 2025 in
Pay section);
BLS ECEC (private industry, June 2022): benefits = 29.5% of total
compensation => loaded multiplier 1/(1-0.295) = 1.418;
SWE-bench Verified annotated time-to-fix (Epoch AI / OpenAI
annotation): <15 min 39%, 15 min-1 h 52%, 1-4 h 8%, >4 h 3 issues
(METR: estimates likely optimistic);
OpenRouter public price list (per million tokens) for k; published
agent cost per task in SWE-bench submission READMEs ($0.60 Bracket,
$0.64 Moatless Claude-4-Sonnet, $0.84 Cortexa)."""
import numpy as np
out = []; P = out.append
wage = 64.44; mult = 1/(1-0.295); loaded = wage*mult
P(f"Loaded hourly cost of a software developer: ${wage:.2f}/h x
{mult:.3f} = ${loaded:.0f}/h")
shares = [(.39, 10/60), (.52, .5), (.08, 2.0), (.006, 6.0)]
# (share, representative hours; bucket midpoints)
mean_h = sum(s*h for s, h in shares)/sum(s for s, h in shares)
P(f"SWE-bench Verified expected hands-on time (annotated, familiar
engineer): {mean_h:.2f} h => H = ${mean_h*loaded:.0f}; unfamiliar-
engineer multiplier 2x-4x => H = ${2*mean_h*loaded:.0f}-
${4*mean_h*loaded:.0f}")
P(f"Bucket values: <15min => ${10/60*loaded:.0f}; 15-60min =>
${0.5*loaded:.0f}; 1-4h => ${2*loaded:.0f}-${4*loaded:.0f}")
# k from token usage x OpenRouter prices
def k_attempt(pt, ct, pin, pout): return (pt*pin + ct*pout)/1e6
prices = {"Gemma-3-4B (small)": (.04, .05), "Qwen3-Coder-30B-A3B":
(.07, .28), "Qwen3-Coder": (.30, 1.0), "Claude Sonnet 5.5": (2., 10.),
"o4-mini (reasoning)": (1.1, 4.4)}
tok = {"function-level, non-reasoning (RepairBench: ~1,700 in / ~800
out)": (1700, 800), "function-level, reasoning (RepairBench o4-mini:
~1,400 in / ~10,400 out)": (1420, 10408)}
P("\nPer-attempt vendor cost k for function-level repair (price list x
observed tokens):")
for tname, (pt, ct) in tok.items():
    P(f"  {tname}")
    for m, (pi, po) in prices.items():
        P(f"     {m:24s} k = ${k_attempt(pt, ct, pi, po):.5f}")
P("Repository-level agentic repair: published cost per task $0.60-
$0.84 (one trajectory or one multi-sample pipeline)")
P("Verification cost v per attempt: automated tests only (CI run) ~
$0.01-$0.50; with 2-10 min of manual triage ~ $3-$15 (at $91/h)")
np.save("calibration_loaded.npy", np.array([loaded, mean_h]))
```

```
open("results_calibration.txt", "w").write("\n".join(out)); print("\
n".join(out))
```

sensitivity.py

Sensitivity across calibrated ranges (Table 5).

```
"""Sensitivity of the main conclusions over calibrated ranges of (H,
v, k) and three heterogeneity profiles. Exact enumeration of budget
steps (no price grids)."""
import numpy as np, itertools
from scipy.special import betaln
def stats(a, b, H, v, k, J=2000):
    n = np.arange(J+1); S = np.exp(betaln(a, b+n) - betaln(a, b)); E =
np.concatenate([[0], np.cumsum(S)])[:J+1]; Pn = 1 - S
    q = a/(a+b+np.arange(J))
    W = H*Pn - (k+v)*E
    fb = int(np.argmax(W[:J])) if (q[0]*H > k+v) else 0
    res = dict(NFB=fb, SFB=W[fb] if fb else 0.0)
    if fb == 0: return res
    nmax = min(int(a*H/v)+2, J-1)
    best = {"A": (-1, 0, 0), "B": (-1, 0, 0)}
# (profit, n, firm saving)
    for m in range(1, nmax+1):
        if q[m-1]*H <= v: break
        wcap = q[m-1]*H - v
        if wcap >= k:
            pr = (wcap - k)*E[m]
            if pr > best["A"][0]: best["A"] = (pr, m, W[m] - pr)
        phi = H - v/q[m-1]
        if phi >= 0:
            pr = phi*Pn[m] - k*E[m]
            if pr > best["B"][0]: best["B"] = (pr, m, W[m] - pr)
    Gmin = v*(Pn[fb]/q[fb-1] - E[fb])
    res.update(A=best["A"], B=best["B"], WA=W[best["A"][1]] if
best["A"][1] else 0, WB=W[best["B"][1]] if best["B"][1] else 0,
rent_share=Gmin/W[fb], W=W)
    return res
profiles = {"RepairBench-like (0.12,0.39)": (.12, .39), "SWE-Verified-
like (0.25,0.34)": (.25, .34), "HumanEvalFix-Llama-like (0.50,0.58)":
(.50, .58)}
regimes = {
 "Function-level, automated verification": dict(H=[15, 30, 50, 100,
200, 400], v=[.01, .03, .1, .3, .5], k=[.0003, .001, .003, .01, .03,
.1]),
 "Function-level, manual triage per attempt": dict(H=[15, 30, 50, 100,
200, 400], v=[3, 5, 8, 15], k=[.0003, .001, .003, .01, .03, .1]),
 "Repository-level agentic": dict(H=[50, 100, 200, 400, 800], v=[.1,
.5, 2, 5], k=[.6, .64, .84, 1.5, 3.]),
}
out = []; P = out.append
for rname, rg in regimes.items():
    P(f"\n=== {rname}:  H in {rg['H']}, v in {rg['v']}, k in
{rg['k']}")
```

```
    P("profile | grid pts | AI worth using (N^FB>=1) | among those:
welfare success fee > per-attempt | vendor profit B>=A | firm saving
B>A | median W/FB: A, B | median N^FB, n_hat(B) | n_hat(B)<N^FB | rent
share of W(N^FB): median [min-max]")
    for pname, (a, b) in profiles.items():
        rows = []; tot = 0
        for H, v, k in itertools.product(rg['H'], rg['v'], rg['k']):
            tot += 1; r = stats(a, b, H, v, k)
            if r['NFB'] >= 1 and r['A'][1] > 0 and r['B'][1] > 0:
rows.append(r)
        if not rows: P(f"{pname} | {tot} | 0%"); continue
        n = len(rows)
        wa = np.array([r['WA']/r['SFB'] for r in rows]); wb =
np.array([r['WB']/r['SFB'] for r in rows])
        P(f"{pname:36s} | {tot:3d} | {100*n/tot:3.0f}% |
{100*np.mean(wb > wa + 1e-9):3.0f}% | {100*np.mean([r['B'][0] >=
r['A'][0] - 1e-9 for r in rows]):3.0f}% | {100*np.mean([r['B'][2] >
r['A'][2] for r in rows]):3.0f}% | {np.median(wa):.2f},
{np.median(wb):.2f} | {np.median([r['NFB'] for r in rows]):.0f},
{np.median([r['B'][1] for r in rows]):.0f} | {100*np.mean([r['B'][1] <
r['NFB'] for r in rows]):3.0f}% | {100*np.median([r['rent_share'] for
r in rows]):.0f}% [{100*min(r['rent_share'] for r in rows):.0f}-
{100*max(r['rent_share'] for r in rows):.0f}%]")
open("results_sensitivity.txt", "w").write("\n".join(out)); print("\
n".join(out))
```

fig_sensitivity.py

Figure 2.

```
import numpy as np, matplotlib; matplotlib.use("Agg"); import
matplotlib.pyplot as plt
from sensitivity import stats
a, b = .12, .39
hv = np.logspace(0.7, 4.3, 40); kv = np.logspace(-3, 1, 40)
# H/v, k/v with v = 1
WA = np.full((len(kv), len(hv)), np.nan); WB = WA.copy(); RS =
WA.copy()
for i, kk in enumerate(kv):
    for j, hh in enumerate(hv):
        r = stats(a, b, hh, 1.0, kk, J=1500)
        if r['NFB'] >= 1 and r.get('A') and r['A'][1] > 0 and r['B']
[1] > 0:
            WA[i, j] = r['WA']/r['SFB']; WB[i, j] = r['WB']/r['SFB'];
RS[i, j] = r['rent_share']
plt.rcParams.update({"font.size": 8.5, "font.family": "serif"})
fig, ax = plt.subplots(3, 1, figsize=(6.3, 8.2),
constrained_layout=True, sharex=True)
for k_, (axx, Z, t) in enumerate(zip(ax, [WA, WB, RS], ["(a) Per-
attempt monopoly: welfare / first best", "(b) Success-fee monopoly:
welfare / first best", "(c) Incentive rent / surplus at first-best
depth"])):
    im = axx.pcolormesh(np.log10(hv), np.log10(kv), Z, vmin=0, vmax=1,
cmap="viridis", shading="auto")
    axx.set_ylabel("log10 (k / v)"); axx.set_title(t, fontsize=9,
loc="left")
    if k_ == 2: axx.set_xlabel("log10 (H / v)   (human repair cost /
firm cost per attempt)")
    fig.colorbar(im, ax=axx, shrink=.9, pad=.02)
fig.savefig("fig_sensitivity.png", dpi=200)
print("saved")
```

verify_proofs.py

Numerical checks of Propositions 4 and 8(b).

```
"""Numerical verification of the rigorous versions of Proposition 4
(integer budget) and Proposition 6(b) (convergence rate)."""
import numpy as np
from scipy.special import betaln
from contracts import Mkt, solve
from diff_bertrand import curves, summarize, share
out = []; P = out.append
def parts(m):
    a, b, H, v, k = m.a, m.b, m.H, m.v, m.k
    phi = lambda n: H - v*(n + a + b)/a                    # fee at
which attempt n+1 becomes unattractive: q_n (H-f) = v
    S = lambda j: m.S[j]; q = lambda n: a/(a+b+n)
    L = lambda n: phi(n)*m.P(n) - k*m.att(n)                # profit
at left end of step n (attained)
    R = lambda n: phi(n-1)*m.P(n) - k*m.att(n)              # sup of
profit within step n (approached from below)
    return phi, S, q, L, R
# ---------- (1) Proposition 4: identities and the dominance chain
----------
rng = np.random.default_rng(21); bad = dict(identity=0, sign=0,
chain=0, argmax=0, n=0, strict_under=0, in_step=0)
for t in range(300):
    a = rng.uniform(.05, .6); b = np.exp(rng.uniform(np.log(.1),
np.log(4))); H = np.exp(rng.uniform(np.log(30), np.log(500)))
    v = H*np.exp(rng.uniform(np.log(.001), np.log(.03))); k =
v*np.exp(rng.uniform(np.log(.1), np.log(3)))
    m = Mkt(a, b, H, v, k, J=3000); fb = m.first_best(); n0 = fb['N'];
fe = H*k/(k+v)
    if n0 < 1: continue
    bad['n'] += 1; phi, S, q, L, R = parts(m)
    top = min(m.Nstar(H/v), 2999) + 1
    for n in range(1, min(top, 400)):
        D = R(n) - L(n-1) if n-1 >= 0 else None
        pred = S(n-1)*(phi(n-1)*q(n-1) - k)
        if abs(D - pred) > 1e-7*max(1, abs(pred)): bad['identity'] +=
1
        if (D > 1e-9) != (phi(n-1) > fe + 1e-9) and abs(D) > 1e-9:
bad['sign'] += 1
    # chain: R(n) < R(n0) for all n > n0
    for n in range(n0+1, min(top, n0+200)):
        if not R(n) < R(n0) + 1e-9: bad['chain'] += 1
    # argmax of R over n equals brute-force optimum budget (success-
fee monopoly)
    nmax = int(min(m.Nstar(H/v), 2999))
    Rs = [(R(n), n) for n in range(1, nmax+1)]
    nhat = max(Rs)[1]
    fs = np.linspace(0, H, 4000); prof = [m.success_fee(f)['vend'] for
```

```
f in fs]; bf_budget = m.success_fee(fs[int(np.argmax(prof))])['N']
    if abs(bf_budget - nhat) > 1 and not (R(nhat) - max(prof) > -1e-
3*max(1, max(prof))): bad['argmax'] += 1
    bad['strict_under'] += nhat < n0; bad['in_step'] += nhat == n0
P("== Proposition 4 (integer budget): verification ==")
P(f"{bad['n']} draws with N^FB >= 1")
P(f"identity R(n) - L(n-1) = S(n-1)(phi_(n-1) q_(n-1) - k): violations
{bad['identity']}")
P(f"sign(R(n) - L(n-1)) = sign(phi_(n-1) - f^e): violations
{bad['sign']}")
P(f"chain R(n) < R(N^FB) for all n > N^FB: violations {bad['chain']}")
P(f"vendor's best step n_hat = argmax R(n): strictly below N^FB in
{bad['strict_under']} draws, equal to N^FB in {bad['in_step']} draws;
argmax/brute-force mismatches {bad['argmax']}")
# ---------- (2) Proposition 6(b): convergence rate ----------
P("")
P("== Proposition 6(b): convergence in sigma (RepairBench-like, H=100,
v=1, k=0.2, n=5) ==")
m = Mkt(.12, .39, 100., 1., .2); fb = m.first_best(); Gbar =
fb['surplus']
P(f"G_bar <= S^FB = {Gbar:.1f};  rho = exp(G_bar/sigma) - 1;  coverage
limit n/(n+1) = {5/6:.3f}")
P(" sigma | rho  | bound (1-rho)/(1+rho) | A: Pi(p_hat)/Pi_m  N
coverage | B: Pi(p_hat)/Pi_m  N  coverage | monopoly N_A N_B")
cvs = {kk: curves(m, kk) for kk in "AB"}
mono = {}
for kk in "AB":
    ps, F, Pi, N, Wm = cvs[kk]; i = int(np.argmax(Pi)); mono[kk] =
(Pi[i], N[i], ps[i])
for sig in [30., 100., 300., 1000., 3000.]:
    rho = np.exp(Gbar/sig) - 1; row = []
    for kk in "AB":
        r = summarize(m, kk, 5, sig, cvs[kk]);
row.append((r['Pi']/mono[kk][0], r['N'], r['cover']))
    P(f" {sig:6.0f} | {rho:5.2f} | {max((1-rho)/(1+rho), 0):6.3f}
| {row[0][0]:.3f}        {row[0][1]:3d}   {row[0][2]:.3f}    | {row[1]
[0]:.3f}        {row[1][1]:3d}   {row[1][2]:.3f}    | {mono['A']
[1]:3d} {mono['B'][1]:3d}")
open("results_verify_proofs.txt", "w").write("\n".join(out)); print("\
n".join(out))
```

general_theory.py

Distribution-free checks of Propositions 1, 5, 6.

```
"""Distribution-free verification (arbitrary mixing distribution F on
p, e.g., finite mixtures, zero-inflated).
Checks: (1) q_n nonincreasing for any F; (2) incentive-rent theorem:
for every contract (w>=0,f) inducing depth N,
Pi(w,f) <= R(N) - (w/v) G_min(N), with R(N)=W(N)-G_min(N),
G_min(N)=v[P(N)/q_{N-1}-E(N)];
(3) increment formula; (4) the vendor's optimum over the 2-parameter
class has w=0 (pure success fee); (5) lump-sum fee => first best."""
import numpy as np
rng = np.random.default_rng(99); out = []; P_ = out.append
JJ = 400
class Gen:
    def __init__(s, ps, ws):
        s.ps, s.ws = np.array(ps), np.array(ws)/np.sum(ws); n =
np.arange(JJ+1)
        s.S = (s.ws[:, None]*(1-s.ps[:, None])**n[None, :]).sum(0)
# S(j)=E[(1-p)^j]
        s.q = np.array([ (s.ws*s.ps*(1-s.ps)**j).sum()/s.S[j] if
s.S[j] > 1e-300 else 0 for j in range(JJ)])
        s.cum = np.concatenate([[0], np.cumsum(s.S)])
    def E(s, N): return s.cum[N]
    def Pr(s, N): return 1 - s.S[N]
def depth(g, w, f, v, H):                # firm continues at n iff q_n
(H-f) > w+v (stop when indifferent)
    idx = np.nonzero(g.q*(H-f) <= w+v)[0]; return int(idx[0]) if
len(idx) else JJ-1
viol = dict(q=0, bound=0, incr=0, n=0, wpos=0, lump=0)
best_w0 = 0; tested = 0; n1 = 0; nbad = 0; ubviol = 0; gaps = []
for t in range(200):
    kmix = rng.integers(2, 6); ps = rng.uniform(0, 1,
kmix)**rng.uniform(1, 4)
    if rng.random() < .4: ps[0] = 0.0                                  #
unsolvable atom
    ws = rng.dirichlet(np.ones(kmix)); g = Gen(ps, ws)
    H = np.exp(rng.uniform(np.log(30), np.log(300))); v =
H*np.exp(rng.uniform(np.log(.003), np.log(.03))); k =
v*np.exp(rng.uniform(np.log(.2), np.log(2)))
    if (np.diff(g.q[:100]) > 1e-12).any(): viol['q'] += 1
    W = lambda N: H*g.Pr(N) - (k+v)*g.E(N)
    Gmin = lambda N: v*(g.Pr(N)/g.q[N-1] - g.E(N)) if N >= 1 and
g.q[N-1] > 0 else np.nan
    Rf = lambda N: W(N) - Gmin(N)
    # (2) random contracts
    for _ in range(300):
        w = rng.uniform(0, H*.5); f = rng.uniform(0, H); N = depth(g,
w, f, v, H)
        if N < 1 or g.q[N-1] <= 0: continue
```

```
        Pi = (w-k)*g.E(N) + f*g.Pr(N)
        viol['n'] += 1
        if Pi > Rf(N) - (w/v)*Gmin(N) + 1e-7*max(1, abs(Pi)):
viol['bound'] += 1
    # (3) increment formula
    for N in range(1, 40):
        if g.q[N] <= 1e-12 or g.q[N-1] <= 1e-12: break
        lhs = Rf(N+1) - Rf(N); rhs = g.S[N]*(g.q[N]*H - (k+v)) -
v*g.Pr(N)*(1/g.q[N] - 1/g.q[N-1])
        if abs(lhs - rhs) > 1e-7*max(1, abs(rhs)): viol['incr'] += 1
    # (4) vendor optimum over (w,f) grid has w ~ 0
    best = (-1e9, None)
    for w in np.linspace(0, H*.4, 60):
        for f in np.linspace(0, H, 120):
            N = depth(g, w, f, v, H)
            if N < 1: continue
            if v*g.E(N) + (w*g.E(N) + f*g.Pr(N)) + H*(1-g.Pr(N)) > H +
1e-9: continue
            Pi = (w-k)*g.E(N) + f*g.Pr(N)
            if Pi > best[0]: best = (Pi, (w, f, N))
    Rmax = max(Rf(N) for N in range(1, JJ-1) if g.q[N-1] > 1e-12 and
not np.isnan(Rf(N))) if (g.q[:JJ-2] > 1e-12).any() else None
    if best[1] is not None and Rmax is not None:
        bad = best[0] > Rmax + 1e-7*max(1, abs(Rmax)); ubviol += bad
        if bad: print('VIOL', best, Rmax, 'qN', g.q[best[1][2]-1],
'ps', np.round(g.ps,3), 'ws', np.round(g.ws,2), 'H,v,k', round(H,1),
round(v,2), round(k,2))
        gaps.append((Rmax - best[0])/max(abs(Rmax), 1e-9))
    if best[1] is not None:
        tested += 1; ok = best[1][0] <= H*.4/59 + 1e-9; best_w0 += ok
or best[1][2] == 1; n1 += (not ok) and best[1][2] == 1; nbad += (not
ok) and best[1][2] > 1                  # w within one grid step of 0
    # (5) lump sum: w=k, F=S^FB -> first best
    Nfb = int(np.nonzero(g.q*H <= k+v)[0][0]) if (g.q*H <= k+v).any()
else JJ-1
    N_k = depth(g, k, 0, v, H)
    if N_k != Nfb: viol['lump'] += 1
P_(f"200 random heterogeneity distributions (finite mixtures with 2-5
atoms, 40% with an unsolvable atom p=0)")
P_(f"(1) q_n nonincreasing: violations {viol['q']}")
P_(f"(2) Pi(w,f) <= R(N) - (w/v) G_min(N): {viol['bound']} violations
in {viol['n']} contracts")
P_(f"(3) increment formula R(N+1)-R(N) = S(N)[q_N H-(k+v)] - v P(N)
(1/q_N - 1/q_(N-1)): violations {viol['incr']}")
P_(f"(4) vendor's grid-optimal contract has per-attempt fee within one
grid step of 0, or induces a single attempt (where G_min=0 and the
forms tie), in {best_w0} of {tested} distributions; of the exceptions
to w=0, {n1} are single-attempt ties and {nbad} are other")
P_(f"(5) w=k implements first-best depth under every distribution:
violations {viol['lump']}")
```

```
P_(f"(6) grid-optimal profit never exceeds max_N R(N) (violations
{ubviol}); relative gap to max_N R(N): median
{np.median(gaps)*100:.2f}%, max {np.max(gaps)*100:.2f}% (grid
resolution)")
open("results_general_theory.txt", "w").write("\n".join(out));
print("\n".join(out))
```

load_repairbench.py

RepairBench: extract per-bug success counts.

```
"""Extract per-bug (n, k) plausible-patch counts and token usage from
RepairBench results."""
import json, os, glob, sys
ROOT = sys.argv[1] if len(sys.argv) > 1 else
"<REPAIRBENCH_CLONE>/results"
rows = []
for mdir in sorted(glob.glob(f"{ROOT}/*/")):
    model = os.path.basename(mdir.rstrip("/"))
    for ds in ["defects4j", "gitbugjava"]:
        for f in glob.glob(f"{mdir}{ds}/evaluation_*.jsonl"):
            for line in open(f):
                x = json.loads(line)
                ev, gen = x.get("evaluation"), x.get("generation")
                if not ev: continue
                ok = [bool(e) and str(e.get("test")) == "True" for e
in ev]
                toks = []
                if isinstance(gen, list):
                    for g in gen:
                        u = (g or {}).get("usage") or {}
                        toks.append((u.get("prompt_tokens") or 0,
u.get("completion_tokens") or 0))
                rows.append(dict(model=model, dataset=ds,
bug=x["identifier"], n=len(ok), k=sum(ok),
                                 prompt_tok=sum(t[0] for t in toks) /
max(len(toks), 1),
                                 comp_tok=sum(t[1] for t in toks) /
max(len(toks), 1)))
json.dump(rows, open("bug_counts.json", "w"))
print(len(rows), "bug-model records;", len({r['model'] for r in
rows}), "models")
from collections import Counter
print(Counter(r['n'] for r in rows).most_common(6))
```

analyze.py

RepairBench: model fits and out-of-sample policy comparison (Section 6).

```
"""Empirical test of the optimal-stopping model on RepairBench (n=10
samples per bug, test-pass = success).
Models: Binomial (homogeneous p), Beta-Binomial (heterogeneous), Zero-
inflated Beta-Binomial (unsolvable mass)."""
import json, numpy as np, hashlib
from scipy.optimize import minimize
from scipy.special import betaln, gammaln, expit
rows = [r for r in json.load(open("bug_counts.json")) if r["n"] == 10]
N = 10
lchoose = lambda n, k: gammaln(n+1) - gammaln(k+1) - gammaln(n-k+1)

def ll_bin(k, p): return lchoose(N, k) + k*np.log(p) + (N-
k)*np.log1p(-p)
def ll_bb(k, a, b): return lchoose(N, k) + betaln(k+a, N-k+b) -
betaln(a, b)
def ll_zi(k, a, b, pi):
    base = np.exp(ll_bb(k, a, b)); return np.log(pi*(k == 0) + (1-
pi)*base)

def fit_bin(k): p = np.clip(k.mean()/N, 1e-6, 1-1e-6); return p,
ll_bin(k, p).sum()
def fit_bb(k):
    f = lambda t: -ll_bb(k, *np.exp(t)).sum()
    best = min((minimize(f, np.log(s), method="Nelder-Mead") for s in
([.5, 1.], [1., 3.], [.2, .5])), key=lambda r: r.fun)
    return np.exp(best.x), -best.fun
def fit_zi(k):
    f = lambda t: -ll_zi(k, np.exp(t[0]), np.exp(t[1]),
expit(t[2])).sum()
    best = min((minimize(f, [np.log(s[0]), np.log(s[1]), s[2]],
method="Nelder-Mead", options=dict(maxiter=2000)) for s in ([.8, 1.5,
0.], [2., 3., -1.], [.5, .5, 1.])), key=lambda r: r.fun)
    return (np.exp(best.x[0]), np.exp(best.x[1]), expit(best.x[2])), -
best.fun

def nstar_bb(a, b, r, cap=100000):  # attempts allowed; r = H/c
    return int(max(0, np.ceil(a*r - a - b)))
def nstar_zi(a, b, pi, r, cap=5000):
    n = np.arange(cap)
    S = np.exp(betaln(a, b+n) - betaln(a, b))
    w = (1-pi)*S / (pi + (1-pi)*S)
    q = w * a/(a+b+n)
    idx = np.nonzero(q*r <= 1)[0]; return int(idx[0]) if len(idx) else
cap

def emp_cost(ks, j, r):  # exact expected cost (in units of c) using
exchangeability of the 10 samples; cap j<=10
```

```
    pfail = lambda i: np.array([np.exp(lchoose(N-k, i) - lchoose(N,
i)) if N-k >= i else 0.0 for k in ks])
    att = sum(pfail(i) for i in range(j))          # E[min(T,j)]
    return float((att + r*pfail(j)).mean())

out = []
P = out.append
by = {}
for r_ in rows: by.setdefault((r_["model"], r_["dataset"]),
[]).append(r_)
res = []
for (m, ds), L in sorted(by.items()):
    if len(L) < 80: continue
    k = np.array([x["k"] for x in L])
    # deterministic 50/50 split by bug id hash
    tr = np.array([int(hashlib.md5(x["bug"].encode()).hexdigest(), 16)
% 2 == 0 for x in L])
    p, l0 = fit_bin(k); (a, b), l1 = fit_bb(k); (az, bz, pz), l2 =
fit_zi(k)
    # held-out log-lik per bug
    pt, _ = fit_bin(k[tr]); (at, bt), _ = fit_bb(k[tr]); (azt, bzt,
pzt), _ = fit_zi(k[tr])
    te = ~tr
    res.append(dict(model=m, ds=ds, nb=len(L), mean_succ=k.mean()/N,
frac0=(k == 0).mean(), a=a, b=b, az=az, bz=bz, pz=pz,
        aic_bin=2-2*l0, aic_bb=4-2*l1, aic_zi=6-2*l2,
        te_bin=ll_bin(k[te], pt).mean(), te_bb=ll_bb(k[te], at,
bt).mean(), te_zi=ll_zi(k[te], azt, bzt, pzt).mean()))
json.dump(res, open("fits.json", "w"))
R = res
P(f"Fitted (model x dataset) combos: {len(R)}  (n=10 samples/bug, >=80
bugs)")
P(f"Beta-Binomial beats Binomial by AIC: {sum(x['aic_bb'] <
x['aic_bin'] for x in R)}/{len(R)};  held-out loglik: {sum(x['te_bb']
> x['te_bin'] for x in R)}/{len(R)}")
P(f"Zero-inflated beats Beta-Binomial by AIC: {sum(x['aic_zi'] <
x['aic_bb'] for x in R)}/{len(R)};  held-out loglik: {sum(x['te_zi'] >
x['te_bb'] for x in R)}/{len(R)}")
P(f"Median fraction of bugs with 0/10 successes:
{np.median([x['frac0'] for x in R]):.2f};  median mean success rate:
{np.median([x['mean_succ'] for x in R]):.3f}")
P(f"Median fitted (a, b) Beta-Binomial: a={np.median([x['a'] for x in
R]):.2f}, b={np.median([x['b'] for x in R]):.2f}")
P(f"Median ZI: a={np.median([x['az'] for x in R]):.2f},
b={np.median([x['bz'] for x in R]):.2f}, pi={np.median([x['pz'] for x
in R]):.2f}")
P("")
P("== Reservation threshold N* by H/c (median across combos) ==")
P("H/c   N*_BB(median)  N*_ZI(median)  [N*_BB IQR]")
for r in [5, 10, 30, 100, 300]:
```

```
    nb_ = [nstar_bb(x['a'], x['b'], r) for x in R]; nz =
[nstar_zi(x['az'], x['bz'], x['pz'], r) for x in R]
    P(f"{r:4d}  {np.median(nb_):8.0f}  {np.median(nz):12.0f}
[{np.percentile(nb_,25):.0f}, {np.percentile(nb_,75):.0f}]")

# ---- out-of-sample policy evaluation on held-out bugs, caps within
observed range (<=10) ----
P("")
P("== Held-out cost (units of c) per bug, caps restricted to 0..10
(exact, from real outcomes) ==")
P("H/c  policy                              mean cost over combos   %vs
human-only   best-in-hindsight")
for r in [3, 5, 8, 10]:
    acc = {"human-only": [], "fixed k=1": [], "fixed k=3": [], "fixed
k=5": [], "fixed k=10": [], "BB reservation (train fit)": [], "ZI
reservation (train fit)": [], "hindsight best fixed": [], "train-best
fixed": []}
    for (m, ds), L in sorted(by.items()):
        if len(L) < 80: continue
        k = np.array([x["k"] for x in L]); tr =
np.array([int(hashlib.md5(x["bug"].encode()).hexdigest(), 16) % 2 == 0
for x in L]); te = ~tr
        (at, bt), _ = fit_bb(k[tr]); (azt, bzt, pzt), _ =
fit_zi(k[tr])
        cost = lambda j: emp_cost(k[te], j, r)
        acc["human-only"].append(cost(0))
        for j in (1, 3, 5, 10): acc[f"fixed k={j}"].append(cost(j))
        acc["BB reservation (train fit)"].append(cost(min(10,
nstar_bb(at, bt, r))))
        acc["ZI reservation (train fit)"].append(cost(min(10,
nstar_zi(azt, bzt, pzt, r))))
        acc["hindsight best fixed"].append(min(cost(j) for j in
range(11)))
        acc["train-best
fixed"].append(cost(int(np.argmin([emp_cost(k[tr], j, r) for j in
range(11)]))))
    base = np.mean(acc["human-only"])
    for kk, v in acc.items(): P(f"{r:4d}  {kk:30s}  {np.mean(v):10.3f}
{100*(1-np.mean(v)/base):6.1f}%")
    P("")
# ---- cross-model heterogeneity in implied N* & token cost ----
tok = {}
for x in rows: tok.setdefault(x["model"], []).append(x["comp_tok"])
P("== Selected models: fits (all bugs) ==")
sel = ["o4-mini-2025-04-16-high", "deepseek-v3", "gpt-4o-2024-11-20",
"qwen-2.5-coder-32b-instruct", "llama-3.3-70b-instruct", "claude-3-7-
sonnet-20250219"]
P("model/dataset                          nbugs  mean_succ  frac0   a
b     N*_BB(H/c=30)  N*_BB(100)  pi_ZI  N*_ZI(30)  N*_ZI(100)")
for x in R:
```

```
    if x['model'] in sel:
        P(f"{x['model'][:28]+'/'+x['ds']:40s} {x['nb']:4d}
{x['mean_succ']:8.3f}  {x['frac0']:5.2f} {x['a']:6.2f} {x['b']:6.2f}
{nstar_bb(x['a'],x['b'],30):8d}  {nstar_bb(x['a'],x['b'],100):9d}
{x['pz']:5.2f}  {nstar_zi(x['az'],x['bz'],x['pz'],30):6d}
{nstar_zi(x['az'],x['bz'],x['pz'],100):7d}")
open("results_empirical.txt", "w").write("\n".join(out))
print("\n".join(out))
```

swebench_repo_level.py

SWE-bench: calibration from published coverage points (Section 6).

```
"""Repository-level (SWE-bench) extension.
(A) Calibrate Beta-geometric (a,b) from published pass@1 / pass@k
pairs (aggregate coverage curves).
(B) Cross-submission heterogeneity of instance difficulty from
official results (proxy, NOT attempt-level)."""
import json, glob, os, sys, numpy as np
from scipy.optimize import brentq, minimize
from scipy.special import betaln, gammaln
ROOT = sys.argv[1] if len(sys.argv) > 1 else
"<SWE_BENCH_EXPERIMENTS_CLONE>/evaluation"
out = []; P = out.append

cov = lambda a, b, k: 1 - np.exp(betaln(a, b+k) - betaln(a, b))
def calib(m, k, ck):                          # mean=m (pass@1), coverage
at k = ck
    f = lambda la: cov(np.exp(la), np.exp(la)*(1-m)/m, k) - ck
    a = np.exp(brentq(f, -12, 6)); return a, a*(1-m)/m
nstar = lambda a, b, r: int(max(0, np.ceil(a*r - a - b)))
P("== (A) Calibration from published coverage points (aggregate; SWE-
bench Verified=500 tasks unless noted) ==")
P("source                                    pass@1  pass@k   k     a
b     N*(H/c=30) N*(100) N*(300) N*(1000)")
pts = [("DeepSWE-Preview (Qwen3-32B), Verified", .422, 16, .710),
       ("Cortexa o3, Verified (pass@7=75%)", None, 7, .75),
       ("Large Language Monkeys, Lite, DS-Coder-V2", .159, 250, .56)]
for name, m, k, ck in pts:
    if m is None: P(f"{name:40s}  (pass@1 not stated in repo README;
skipped)"); continue
    a, b = calib(m, k, ck)
    P(f"{name:40s}  {m:5.3f}  {ck:5.3f} {k:4d}  {a:5.3f}  {b:6.3f}
{nstar(a,b,30):6d}  {nstar(a,b,100):6d}  {nstar(a,b,300):6d}
{nstar(a,b,1000):6d}")
    ks = [1, 2, 4, 8, 16, 32, 64, 128, 250]
    P("    implied coverage: " + ", ".join(f"@{kk}={cov(a,b,kk):.2f}"
for kk in ks))

P("")
P("== (B) Cross-submission instance heterogeneity (proxy; systems
differ, not iid samples) ==")
for split, nexp in [("lite", 300), ("verified", 500)]:
    resolved, nsub = {}, 0
    for f in glob.glob(f"{ROOT}/{split}/*/results/results.json"):
        try: d = json.load(open(f))
        except Exception: continue
        r = d.get("resolved")
        if not r: continue
        nsub += 1
```

```
        for i in r: resolved[i] = resolved.get(i, 0) + 1
    ids = set(resolved)
    k = np.array(list(resolved.values()))
    never = nexp - len(ids)
    P(f"{split}: {nsub} submissions; instances resolved by >=1
submission: {len(ids)}/{nexp}; never resolved: {never}
({100*never/nexp:.1f}%)")
    q = np.percentile(k/nsub, [10, 25, 50, 75, 90]); P(f"   share-of-
systems resolving an instance (resolved-by-some instances) percentiles
10/25/50/75/90: {np.round(q,2)}")
    # Beta-binomial over all instances incl. zeros, n = nsub
    kk = np.concatenate([k, np.zeros(never, int)]); n = nsub
    lc = gammaln(n+1) - gammaln(kk+1) - gammaln(n-kk+1)
    nll = lambda t: -(lc + betaln(kk+np.exp(t[0]), n-kk+np.exp(t[1]))
- betaln(np.exp(t[0]), np.exp(t[1]))).sum()
    r_ = minimize(nll, np.log([.5, .5]), method="Nelder-Mead"); a, b =
np.exp(r_.x)
    P(f"   Beta fit across systems: a={a:.2f}, b={b:.2f} (U-shaped if
both<1), mean={a/(a+b):.2f}")
open("results_swebench.txt", "w").write("\n".join(out)); print("\
n".join(out))
```

sample_humanevalfix.py

HumanEvalFix: sampling and test execution (Section 6).

```
"""Sample n patches per HumanEvalFix(Python) bug from an OpenRouter
model, run tests, log (success, tokens, cost).
Usage: python3 sample_humanevalfix.py MODEL N_SAMPLES [N_TASKS]"""
import sys, os, re, json, subprocess, tempfile, concurrent.futures as
cf, urllib.request, time
import pandas as pd
MODEL, NS = sys.argv[1], int(sys.argv[2]); NT = int(sys.argv[3]) if
len(sys.argv) > 3 else 164
OUT = f"raw_{MODEL.replace('/', '_')}.jsonl"
df = pd.read_parquet("hep_python.parquet").iloc[:NT]
done = set()
if os.path.exists(OUT):
    for l in open(OUT): r = json.loads(l); done.add((r["task"],
r["i"]))

def prompt(r):
    return (f"Fix bugs in {r['entry_point']}.\n\n{r['declaration']}
{r['buggy_solution']}\n\n"
            f"The function must satisfy its specification:\
n{r['docstring']}\n\n"
            "Return the full corrected function in a single ```python
code block, nothing else.")

def call(r):
    body = json.dumps({"model": MODEL, "messages": [{"role": "user",
"content": prompt(r)}], "temperature": 1.0, "max_tokens": 1500,
"usage": {"include": True}}).encode()
    for attempt in range(5):
        try:
            headers = {"Content-Type": "application/json"}
            if os.environ.get("OPENROUTER_API_KEY"):
headers["Authorization"] = "Bearer " +
os.environ["OPENROUTER_API_KEY"]
            req =
urllib.request.Request("https://openrouter.ai/api/v1/chat/completions"
, body, headers)
            return json.load(urllib.request.urlopen(req, timeout=120))
        except Exception as e:
            time.sleep(2 ** attempt)
    return None

def extract(txt):
    m = re.findall(r"```(?:python)?\n(.*?)```", txt or "", re.S)
    return m[0] if m else (txt or "")

def run_tests(code, r):
    prog = f"{r['import']}\n{code}\n\n{r['test_setup']}\n{r['test']}\
```

```
ncheck({r['entry_point']})\n"
    with tempfile.TemporaryDirectory() as d:
        p = os.path.join(d, "t.py"); open(p, "w").write(prog)
        try:
            return subprocess.run([sys.executable, "-I", p], cwd=d,
capture_output=True, timeout=10, stdin=subprocess.DEVNULL).returncode
== 0
        except Exception: return False

def job(args):
    ti, i = args; r = df.iloc[ti]
    resp = call(r)
    if not resp or "choices" not in resp: return
dict(task=r["task_id"], i=i, ok=False, error=True)
    txt = resp["choices"][0]["message"].get("content") or ""
    u = resp.get("usage", {})
    return dict(task=r["task_id"], i=i, ok=run_tests(extract(txt), r),
pt=u.get("prompt_tokens"), ct=u.get("completion_tokens"),
cost=u.get("cost"), error=False)

jobs = [(t, i) for t in range(len(df)) for i in range(NS) if
(df.iloc[t]["task_id"], i) not in done]
print(len(jobs), "jobs", flush=True)
with cf.ThreadPoolExecutor(16) as ex, open(OUT, "a") as f:
    for n, res in enumerate(ex.map(job, jobs)):
        f.write(json.dumps(res) + "\n"); f.flush()
        if n % 500 == 0: print(n, flush=True)
```

analyze_hef.py

HumanEvalFix: analysis of 40 samples per bug (Section 6).

```
"""HumanEvalFix(Python): n=40 samples/bug from 3 small models. Tests
(i) heterogeneity, (ii) reservation rule vs fixed caps up to 40, (iii)
calibration of the extrapolation from the first 10 samples to caps up
to 40."""
import json, glob, hashlib, numpy as np
from scipy.optimize import minimize
from scipy.special import betaln, gammaln, comb, expit
NS = 40
lc = lambda n, k: gammaln(n+1) - gammaln(k+1) - gammaln(n-k+1)
def ll_bb(k, n, a, b): return lc(n, k) + betaln(k+a, n-k+b) -
betaln(a, b)
def ll_bin(k, n, p): return lc(n, k) + k*np.log(p) + (n-k)*np.log1p(-
p)
def fit_bb(k, n):
    f = lambda t: -ll_bb(k, n, *np.exp(t)).sum()
    r = min((minimize(f, np.log(s), method="Nelder-Mead") for s in
([.5, 1.], [1., 3.], [.2, .5], [2., .5])), key=lambda r: r.fun)
    return np.exp(r.x), -r.fun
nstar = lambda a, b, r: int(max(0, np.ceil(a*r - a - b)))
def ll_zi(k, n, a, b, pi): return np.log(pi*(k == 0) + (1-
pi)*np.exp(ll_bb(k, n, a, b)))
def fit_zi(k, n):
    f = lambda t: -ll_zi(k, n, np.exp(t[0]), np.exp(t[1]),
expit(t[2])).sum()
    r = min((minimize(f, [np.log(s[0]), np.log(s[1]), s[2]],
method="Nelder-Mead", options=dict(maxiter=3000)) for s in ([.8, 1.5,
0.], [2., 3., -1.], [.5, .5, 1.], [.5, .1, -2.])), key=lambda r:
r.fun)
    return (np.exp(r.x[0]), np.exp(r.x[1]), expit(r.x[2])), -r.fun
def nstar_zi(a, b, pi, r, cap=5000):
    n = np.arange(cap); S = np.exp(betaln(a, b+n) - betaln(a, b)); w =
(1-pi)*S/(pi + (1-pi)*S)
    idx = np.nonzero(w*a/(a+b+n)*r <= 1)[0]; return int(idx[0]) if
len(idx) else cap
def pfail(k, n, j):  # P(no success in first j of n exchangeable
samples | k successes)
    return np.where(n-k >= j, np.exp(lc(n-k, j) - lc(n, j)), 0.0) if
True else None
def lcomb(n, k): return gammaln(n+1)-gammaln(k+1)-gammaln(n-k+1)
def pf(k, n, j):
    k = np.asarray(k); out = np.zeros(k.shape)
    m = (n-k) >= j; out[m] = np.exp(lcomb(n-k[m], j) - lcomb(n, j));
return out
def cost(k, n, j, r): return float((sum(pf(k, n, i) for i in range(j))
+ r*pf(k, n, j)).mean())
out = []; P = out.append
for f in sorted(glob.glob("raw_*.jsonl")):
```

```
    model = f[4:-6]; per = {}
    for l in open(f):
        x = json.loads(l)
        if x.get("error"): continue
        per.setdefault(x["task"], []).append(bool(x["ok"]))
    tasks = {t: v for t, v in per.items() if len(v) >= NS}
    if len(tasks) < 100: P(f"{model}: only {len(tasks)} complete
tasks; skipped"); continue
    ks = np.array([sum(v[:NS]) for v in tasks.values()]); k10 =
np.array([sum(v[:10]) for v in tasks.values()])
    ids = list(tasks)
    P(f"=== {model}: {len(tasks)} bugs x {NS} samples; mean success
{ks.mean()/NS:.3f}; share with 0/{NS}: {(ks==0).mean():.2f}; share
with {NS}/{NS}: {(ks==NS).mean():.2f}")
    p = np.clip(ks.mean()/NS, 1e-6, 1-1e-6); (a, b), l1 = fit_bb(ks,
NS); l0 = ll_bin(ks, NS, p).sum()
    (az, bz, pz), l2 = fit_zi(ks, NS)
    P(f"    Beta-binomial a={a:.3f} b={b:.3f}; AIC bin={2-2*l0:.0f} vs
BB={4-2*l1:.0f} vs ZI-BB={6-2*l2:.0f} (ZI: a={az:.3f} b={bz:.3f}
pi={pz:.3f})")
    (a10, b10), _ = fit_bb(k10, 10)
    P(f"    Fit from first 10 samples only: a={a10:.3f} b={b10:.3f}")
    for r in [10, 30, 100, 300]:
        P(f"    H/c={r:4d}: N* BB (n=40)={nstar(a,b,r):3d}  N* BB
(first 10)={nstar(a10,b10,r):3d}  N* ZI
(n=40)={nstar_zi(az,bz,pz,r):3d}")
    # held-out cost vs cap with n=40 data
    tr = np.array([int(hashlib.md5(t.encode()).hexdigest(), 16) % 2 ==
0 for t in ids]); te = ~tr
    P("    Held-out cost (units of c) by H/c: human-only | fixed k=1 |
k=5 | k=10 | BB reservation | ZI reservation | best fixed in
hindsight")
    for r in [10, 30, 100]:
        (at, bt), _ = fit_bb(ks[tr], NS); Nr = min(NS, nstar(at, bt,
r))
        (azt, bzt, pzt), _ = fit_zi(ks[tr], NS); Nz = min(NS,
nstar_zi(azt, bzt, pzt, r))
        hind = min(range(NS+1), key=lambda j: cost(ks[te], NS, j, r))
        P(f"      H/c={r:4d}: {r:6.1f} | {cost(ks[te],NS,1,r):6.2f} |
{cost(ks[te],NS,5,r):6.2f} | {cost(ks[te],NS,10,r):6.2f} |
{cost(ks[te],NS,Nr,r):6.2f} (N={Nr}) | {cost(ks[te],NS,Nz,r):6.2f}
(N={Nz}) | {cost(ks[te],NS,hind,r):6.2f} (k={hind})")
    # calibration of extrapolation: predict coverage at j=40 from
model fit on first 10, compare to observed coverage
    obs = [1 - pf(ks, NS, j).mean() for j in (1, 5, 10, 20, 40)]
    pred = [1 - np.exp(betaln(a10, b10+j) - betaln(a10, b10)) for j in
(1, 5, 10, 20, 40)]
    (az10, bz10, pz10), _ = fit_zi(k10, 10)
    predz = [(1-pz10)*(1 - np.exp(betaln(az10, bz10+j) - betaln(az10,
bz10))) for j in (1, 5, 10, 20, 40)]
```

```
    P("    Coverage at j=1,5,10,20,40  observed: " + ",
".join(f"{x:.3f}" for x in obs) + "   BB first-10 fit: " + ",
".join(f"{x:.3f}" for x in pred) + "   ZI first-10 fit: " + ",
".join(f"{x:.3f}" for x in predz))
    P("")
open("results_hef.txt", "w").write("\n".join(out)); print("\
n".join(out))
```